

Trabajo Práctico: *Memoria*

Sistemas Operativos
Departamento de Computación
FCEyN - UBA
Segundo Cuatrimestre 2026

Introducción

El manejo de memoria es un componente fundamental en el diseño y funcionamiento de los sistemas operativos modernos. A lo largo del tiempo, diversas técnicas han sido desarrolladas para administrar eficientemente el uso de la memoria física. Este trabajo práctico se centra en dos aspectos esenciales de la gestión de memoria: el **manejo de espacio libre** y las **políticas de reemplazo de páginas**.

En la primera parte se abordará cómo los sistemas operativos gestionan el espacio libre dentro de la memoria. Esto incluye el abordaje de problemas como la fragmentación, la eficiencia en tiempo y espacio, y la capacidad de adaptarse a cargas de trabajo variables.

La segunda parte del trabajo se enfoca en las políticas de reemplazo de páginas, un componente crítico de los sistemas de memoria virtual. A través del análisis de distintas políticas de reemplazo, se busca comprender los principios detrás de su diseño, sus ventajas, limitaciones y cómo influyen en el rendimiento general del sistema.

Ambas secciones del trabajo se apoyarán en simuladores que permiten observar y experimentar con el comportamiento de los algoritmos presentados.

Simuladores

Dentro del archivo comprimido del TP encontrarán dos programas de Python: `malloc.py`, que permite observar el funcionamiento de un manejador de memoria o *allocator* simple, y `paging-policy.py`, que permite experimentar con diferentes políticas de reemplazo de páginas.

Para más detalles sobre el uso de cada simulador, consultar los apéndices al final del documento: el **Apéndice A** contiene las instrucciones para el simulador del asignador de memoria, y el **Apéndice B** corresponde al simulador de políticas de reemplazo de páginas.

Nota de Referencia: Los simuladores utilizados en este trabajo práctico corresponden a implementaciones provistas por el libro **Operating Systems: Three Easy Pieces (OSTEP)**, de Remzi H. Arpaci-Dusseau y Andrea C. Arpaci-Dusseau. Se otorgan todos los créditos de la implementación a sus autores.

Consignas

Resolver **grupalmente** los siguientes ejercicios y documentar el proceso de resolución en un informe técnico.

Para cada ejercicio, explicitar el razonamiento utilizado y, cuando resulte pertinente, formular previamente una o más hipótesis o predicciones acerca del compor-

tamiento esperado. Luego, utilizar el simulador para contrastar dichas hipótesis con los resultados obtenidos.

Durante la experimentación, registrar la información necesaria para poder analizar los resultados y extraer conclusiones. Cuando los resultados difieran de las hipótesis planteadas, discutir las posibles causas.

Siempre que resulte pertinente, utilizar tablas, gráficos u otras representaciones que faciliten el análisis y la comparación de los resultados.

Parte 1: Asignación de memoria

1. Ejecutar el simulador con estas opciones: `-n 10 -H 0 -p BEST -s 0` para generar unas pocas operaciones aleatorias. Sin utilizar la opción `-c`, analizar:
 - a) ¿Cuál es el resultado de cada `alloc()/free()`? Observar, por ejemplo, bloque asignado/liberado, su tamaño, su dirección de inicio. ¿Hay operaciones que fallan? ¿Por qué?
 - b) ¿Cuál es el estado de la free list luego de cada operación?
 - c) ¿Qué podés observar de la free list a lo largo del tiempo? Identificar ejemplos concretos de operaciones que aumenten o reduzcan la fragmentación.
2. Repetir el análisis anterior para las políticas de `WORST FIT` y `FIRST FIT`. ¿Qué cambia? Comparar las tres políticas considerando: cantidad de bloques libres, tamaño del bloque libre más grande, fragmentación, cantidad de operaciones que fallan. Explicar, a partir de los resultados observados, por qué las políticas producen comportamientos diferentes.
3. Incrementar la cantidad de asignaciones aleatorias (digamos `-n 1000`) y ejecutar el simulador primero sin coalescing y luego con coalescing (opción `-C`). Analizar el tamaño del bloque libre más grande, la cantidad de bloques libres, la cantidad de operaciones exitosas y fallidas, y todo lo que permita evaluar las diferencias entre ambos enfoques.

Parte 2: Reemplazo de páginas

1. Generar accesos aleatorios con los siguientes argumentos: `-s 0 -n 10`, `-s 1 -n 10`, y `-s 2 -n 10`. Probar con las políticas `FIFO`, `LRU` y `OPT`, sin utilizar la opción `-c`. Para cada traza y cada política, construir una tabla con que muestre el proceso de asignación y remoción de páginas. Calcular la cantidad total de hits, misses y la tasa de misses para cada combinación de traza y política.
2. Considerando 5 marcos de página, generar secuencias de referencias a páginas (al menos 20) que maximicen la cantidad de misses para `FIFO`. Repetir para

LRU. Para la peor secuencia hallada, ¿cuántos marcos de página podrían mejorar el rendimiento y acercarse a OPT?

Sugerencia: utilizar únicamente páginas numeradas entre 0 y 9.

3. Generar una traza de referencias con alguna localidad (temporal, espacial, o ambas). Explicar el procedimiento utilizado para generar la traza, ejecutarla con LRU y señalar ejemplos concretos de localidad en la traza, explicando por qué se comporta de esa manera. Hacer manualmente el seguimiento para la política Second Chance (no incluida en el simulador) sobre la misma traza y comparar el resultado con LRU.

Requerimientos de Entrega

- El trabajo debe entregarse en un único archivo comprimido (.zip o .tar.gz) que contenga:
 - El informe, en formato PDF.
 - Todos los archivos necesarios para reproducir la experimentación incluida en el informe.
- Fecha límite de entrega: **04/10/2026**.
- La entrega se realizará a través de la tarea correspondiente en el campus de la materia.
- **No se aceptarán entregas más allá de la fecha límite, sin excepción.**

Criterios de Evaluación

- El informe deberá ser claro, prolijo y conciso. Se evaluará la calidad y profundidad del contenido por sobre su extensión.
- El informe deberá presentar una estructura y redacción adecuadas para un informe técnico, de manera que el lector pueda comprender el procedimiento, los resultados y las conclusiones.
- El informe deberá explicar y justificar las decisiones tomadas para resolver cada consigna.
- Las respuestas deberán estar fundamentadas mediante el razonamiento y la evidencia obtenida durante la experimentación, y no limitarse a presentar resultados o respuestas finales.

- Las hipótesis o predicciones realizadas deberán ser contrastadas con los resultados obtenidos y las conclusiones deberán ser consistentes con dicho análisis.
- La experimentación deberá estar adecuadamente documentada y ser reproducible: deberán indicarse los parámetros, configuraciones o procedimientos relevantes utilizados para obtener los resultados.
- Los gráficos y tablas deberán estar correctamente identificados y acompañados de una interpretación de la información que presentan. No se considerará suficiente incluirlos sin explicar qué conclusiones permiten obtener.

Bibliografía

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts*. Wiley.
- Tanenbaum, A. S., & Bos, H. (2014). *Modern Operating Systems*. Pearson.
- Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*

A. Simulador de asignación de memoria

Descripción

Este programa permite ver cómo funciona un asignador de memoria simple. Una forma de usarlo es generar operaciones aleatorias de asignación/liberación para luego analizar el aspecto de la *free list*, así como el éxito o el fracaso de cada operación. También puede indicarse una secuencia explícita de operaciones de asignación y liberación de memoria.

Uso

Para ejecutar el programa:

```
prompt> python3 malloc.py
```

Para ver todas las opciones:

```
prompt> python3 malloc.py -h
```

Opciones principales

-s SEED	semilla para la generación aleatoria
-S HEAPSIZE	tamaño del heap
-b BASEADDR	dirección base del heap
-H HEADERSIZE	tamaño del header
-p POLICY	tipo de búsqueda (BEST, WORST, FIRST)
-l ORDER	orden de la lista (ADDRSORT, SIZESORT+, SIZESORT-, INSERT-FRONT, INSERT-BACK)
-C	coalescing de la free list
-n OPSNUM	número de operaciones aleatorias a generar
-r OPSRANGE	tamaño máximo de asignación
-P OPSPALLOC	porcentaje de operaciones que son asignaciones
-A OPSLIST	lista explícita de operaciones (+10,-0,etc)
-c	muestra el resultado de cada operación y el estado de la free list a cada momento

Ejemplo: operaciones aleatorias

```
> python3 malloc.py -S 100 -b 1000 -H 4 -l ADDRSORT -p BEST -n 5
```

Aquí especificamos un heap de 100 bytes (-S 100) que comienza en la dirección 1000 (-b 1000), indicando un adicional de 4 bytes de header por asignación (-H 4).

Especificamos que la free list se ordene por dirección (creciente), establecemos **BEST FIT** como criterio de búsqueda (**-p BEST**), y pedimos que se generen 5 operaciones aleatorias (**-n 5**).

Esta ejecución no mostrará el resultado de cada operación ni cómo se ve la free list. Para ello debe agregarse la opción **-c**.

Ejemplo: lista explícita de operaciones

```
> python3 malloc.py -S 100 -b 1000 -H 4 -l ADDRSORT -p BEST -A +3,-0,+5,-1,+8
```

Este comando establece las mismas condiciones que el ejemplo anterior pero especifica las siguientes operaciones:

- Asignar 3 bytes
- Liberar la primera asignación
- Asignar 5 bytes
- Liberar la segunda asignación
- Asignar 8 bytes

B. Simulador de reemplazo de páginas

Descripción

Este simulador permite experimentar con diferentes políticas de reemplazo de páginas, pudiendo modificar la política y la forma en que se especifican o generan las direcciones.

Uso

Para ejecutar el programa:

```
prompt> python3 paging-policy.py
```

Para ver todas las opciones:

```
prompt> python3 paging-policy.py -h
```

Opciones principales

-a ADDRESSES	un conjunto de páginas a acceder, separadas por comas; -1 para generar aleatoriamente
-f ADDRESSFILE	un archivo con direcciones de páginas
-n NUMADDRS	si la opción -a es -1, es la cantidad de direcciones a generar
-p POLICY	política de reemplazo: FIFO, LRU, OPT (hay otras que no usaremos)
-C CACHESIZE	cantidad de marcos de página disponibles
-m MAXPAGE	si se generan páginas aleatoriamente, es el máximo número de página
-s SEED	semilla para la generación aleatoria
-N	no imprime la traza detallada
-c	muestra si cada solicitud es hit o miss y el estado de la memoria a cada momento

Ejemplo: solicitud de páginas aleatorias

```
> python3 paging-policy.py -s 10 -n 3
```

Aquí especificamos la generación de 3 referencias aleatorias a páginas (-n 3), así como la semilla (-s 10).

Si se agrega la opción -c, se muestra el resultado de cada solicitud y cómo cambia la memoria. La política default es FIFO.

Ejemplo: lista explícita de páginas

```
> python3 paging-policy.py --addresses=0,1,2,0,1,3,0,3,1,2,1  
--policy=LRU -c
```

Este comando muestra el comportamiento de la política LRU para la secuencia de pedidos de página 0 1 2 0 1 3 0 3 1 2 1.